

AccidentGPT: Enhancing Road Safety Through Explainable Artificial Intelligence and Real-Time Predictive Alerting

Abstract

Road traffic accidents are a massive, often overlooked global health crisis. Every year, they claim about 1.19 million lives and leave tens of millions of people injured. It's a tragedy that hits low- and middle-income nations the hardest, and unfortunately, India is consistently one of the most affected areas in the world. Factors like speeding, distracted driving, crumbling infrastructure, unpredictable weather, and the sheer mix of chaotic traffic on our roads keep those numbers far too high.

Most current safety tech—like airbags, anti-lock brakes, or stability control—is fundamentally reactive. These systems are designed to step in only once a crash is already underway. We think it's time to move beyond that. This paper introduces AccidentGPT, a proactive, explainable AI framework built to predict how severe an accident might be before it ever happens. By pinpointing the risks in real time, it gives commuters, drivers, and traffic authorities a chance to actually avoid the collision instead of just surviving it.

To train our system, we gathered five years of police records—spanning 2017 to 2022—covering 12,316 specific accidents across 32 different features. Our pipeline was designed to handle the messy reality of raw data, which meant tackling missing values through imputation, pulling out key features, and

encoding labels before splitting the data 80/20 for testing. Because some severity classes were far rarer than others, we balanced things out using random oversampling.

We ran a comprehensive gauntlet of eleven different classification algorithms, trying out everything from simple linear and distance-based models to complex boosting and kernel-based setups. Two models stood above the rest. The Random Forest Classifier was our top performer in practical terms, hitting 93.36% accuracy with a 0.93 macro-average across precision, recall, and F1-score. We also trained a Convolutional Neural Network that edged it out slightly with 94% accuracy, but that came with a catch: it was much slower to train and had significantly higher latency, making it less ideal for the speed we need.

We leaned on SHAP—SHapley Additive eXplanations—to pull back the curtain on our model's decision-making. Essentially, it assigns a specific contribution score to every feature for any given prediction, letting us see exactly why the system flags a potential danger. When we looked at the big picture, the top five risk factors were clear: the number of vehicles on the road, the time of day, the type of junction, local weather patterns, and the number of casualties involved.

Once the system identifies these risks, a GPT-powered module steps in to do the heavy lifting. It takes those

technical contribution scores and translates them into plain, natural language that a driver can actually understand. That warning is then sent out via a text-to-speech engine. This entire "predict, explain, and communicate" loop turns AccidentGPT into a truly human-centric safety tool. It's built to handle the unique, often unpredictable nature of Indian traffic, but the framework itself is flexible enough to adapt to all sorts of other driving environments, too.

Keywords: *Road Safety · Explainable AI (XAI) · Accident Severity Prediction · SHAP · Random Forest · GPT · Convolutional Neural Network · Natural Language Processing · Indian Road Accident Dataset · Driver Alert System · Machine Learning · Real-Time Safety*

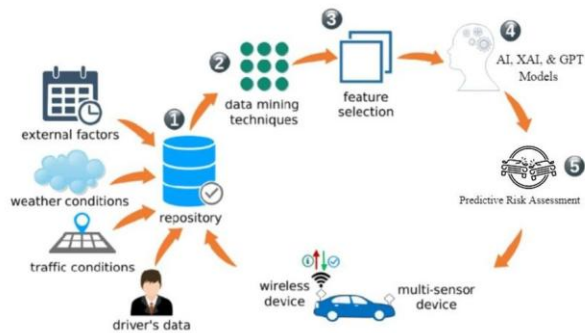


Figure 1 An Overview of the System Architecture

1. INTRODUCTION

Traffic accidents are one of the biggest, most tragic, and entirely preventable public health crises we face today. According to the 2023 WHO Global Status Report, about 1.19 million lives are lost on roads annually, with tens of millions more left with non-fatal injuries. The financial toll is just as devastating; most nations lose roughly 3% of their GDP to these crashes. It's particularly heartbreaking that over 90% of these deaths happen in low- and middle-income countries, which only account for about 60% of the world's vehicles.

In India, the situation is even more dire. Government data consistently shows between 150,000 and 170,000 deaths every year, firmly cementing India's place as one of the world's most dangerous places to drive.

The reasons behind this are deeply layered. On one hand, you have human error—things like speeding, distracted driving, alcohol, or simply skipping seatbelts. But those mistakes aren't happening in a

vacuum. They collide with systemic issues: crumbling infrastructure, faded road markings, and a lack of safe spaces for pedestrians. Then there's the sheer chaos of India's mixed traffic, where high-speed cars share narrow lanes with slow-moving cycle-rickshaws and stray cattle, all while commercial vehicles often operate with little regard for load regulations. It's a complex, dangerous puzzle.

For years, the engineering solution to road safety has been almost entirely passive. We've relied on seat belts, airbags, and crumple zones—things that only do their job once the metal is already folding. Even electronic stability control and ABS are, by design, waiting for trouble to start before they intervene. Sure, these features have saved countless lives in wealthier nations, but they share that same inherent limitation: they're reactive. You also have high-end tech like lane-keeping and automatic emergency braking, but in places like India, that kind of advanced driver assistance is mostly a luxury out of reach for the average motorist.

This is where AI and machine learning bring something entirely new to the table. Instead of waiting for a crash, we can look at the data to predict exactly when and where conditions are becoming dangerous. By training models on years of accident history, we can start to see patterns—how specific weather, road geometry, or time-of-day variables spike the risk of a serious incident. If you can actually run that kind of intelligence in real time, you flip the script. You aren't just protecting a driver during a collision; you're warning them to ease off the throttle, suggesting they avoid a dangerous stretch of road, or even helping traffic police focus their patrols exactly where they're needed most. It moves the safety window from the seconds after a crash to the minutes before one even begins.

Even if we nail the accuracy, turning AI predictions into actual safety measures hits a major wall: the

black-box problem. Most of the powerful models we use—those complex ensembles and deep neural networks—are incredibly opaque. They spit out numbers, but they don't explain the "why." In a field like road safety, where lives are on the line, that mystery isn't just an annoyance; it's a dealbreaker for regulatory approval and driver trust.

Think about it from a driver's perspective. If you suddenly get an alert saying, "High risk: fatal accident probability 0.78," what are you supposed to do with that? Without knowing why, the system reached that conclusion, you have no real basis to decide whether to slow down, pull over, or just ignore it. It's the same for officials. A transport authority can't justify spending limited funds on road infrastructure upgrades if they can't audit the reasoning behind the model's recommendations. You need transparency to turn a cold, clinical prediction into a reliable, actionable tool.

Explainable Artificial Intelligence (XAI) addresses these issues by pairing risk classification outputs with interpretable metrics. Techniques such as SHAP (SHapley Additive eXplanations) and LIME (Local Interpretable Model-agnostic Explanations) provides the users with attribution scores for every prediction for risk classification assessment. It also translates the SHAP metrics into comprehensible English explanations for drive and traffic authorities to act upon.

The paper makes the following key contributions:

- A comprehensive benchmark of eleven diverse machine-learning classifiers on a curated, five-year Indian road accident dataset, establishing Random Forest as the most performant model (93.36% accuracy, macro F1 = 0.93).
- Integration of TreeSHAP for instance level and global feature attribution in the Indian road-safety context, producing the SHAP

analysis on this dataset and identifying most influential risk factors causing the most chaos.

- A novel GPT-to-TTS alert pipeline that transforms technical SHAP analysis into natural-language spoken warnings, tested against real accident scenarios from the dataset.
- A fully specified system architecture with functional and non-functional requirements suitable for real-world deployment, including edge-device and cloud infrastructure specifications.

The remainder of this paper is organized as follows. Section 2 surveys the rapidly evolving landscape of AI-based accident prediction and XAI in transportation. Section 3 describes the dataset, its characteristics, and the preprocessing pipeline in detail. Section 4 presents the machine-learning methodology, model architectures, and evaluation criteria. Section 5 reports experimental results and comparative analysis across all eleven classifiers. Section 6 develops the SHAP explainability analysis in depth. Section 7 describes the GPT-based alert generation and TTS delivery subsystem. Section 8 details the full system architecture and operational requirements. Section 9 discusses real-world applications, limitations, and future research directions. Section 10 concludes the paper.

2. LITERATURE REVIEW

The intersection of machine learning, transportation engineering, and public health has become a crowded and fast-moving field. Researchers have been trying to crack the code on accident prediction for years, moving from simple statistics to ensemble methods, then deep learning, and finally toward the XAI-augmented, multimodal systems we see today.

2.1 Classical Statistical Models

In the early days, researchers relied heavily on regression models. You'd often see Poisson or negative binomial regressions used to map out crash counts, while logistic regression was the go-to for predicting whether a crash was fatal or not. These models were easy to interpret and had a solid mathematical foundation, but they often fell apart when faced with real-world data. They typically assumed that observations were independent—an assumption that rarely holds up on actual roads. Even with "zero-inflated" models designed to account for locations where crashes rarely happen, these methods struggled to capture the messy, non-linear relationships that define real traffic accidents.

Ballamudi (2019) was a pioneer in applying standard ML classifiers—like support vector machines and decision trees—to Indian traffic data. His work proved that these algorithms could actually hold their own on structured datasets, provided the models were tuned to the specific, chaotic reality of Indian roads. He also showed that focusing on features like the time of day and road category was a massive upgrade, a lesson that forms the backbone of the AccidentGPT approach.

2.2 Ensemble and Boosting Methods

As the field matured, ensemble methods started yielding much better results. Researchers like Ahmed et al. (2023) pitted Random Forest, XGBoost, and other boosting techniques against each other using New Zealand's accident data. The Random Forest model came out on top, and by layering in SHAP, they were able to pull back the curtain on why the model was flagging certain accidents as high-risk. This "Random Forest + SHAP" combination has since been validated across China and Turkey, proving that it's incredibly robust, especially when dealing with the mixed data types—numerical and categorical—that usually show up in police records.

Gradient boosting frameworks, like XGBoost, have also made waves. They're exceptionally good at catching the "rare" but devastating events, like fatal crashes, by focusing on error correction. However, the trade-off is often speed; they can be slower to run in

real time compared to a Random Forest, which matters when you're trying to issue a warning before a collision.

2.3 Deep Learning Architectures

Deep learning has surged in popularity since 2018. Some researchers have adapted CNNs—originally designed for images—to work with tabular traffic data by treating the data as 1D sequences. This allows the model to learn complex, local patterns without needing a human to manually engineer every single feature.

Then you have recurrent architectures like LSTMs and GRUs, which are great for looking at time-series data, and even heavy-duty transformer models. Some of these deep models are hitting accuracy rates as high as 98% on large U.S. datasets. But there's a catch: as a recent meta-analysis of 191 studies pointed out, deep learning models often turn into black boxes. While they consistently beat classical models on raw performance, they often sacrifice transparency and are a nightmare to deploy in real-world infrastructure. This tension between performance and interpretability is exactly why we decided to use the Random Forest as our primary production engine, while keeping the CNN as a high-accuracy baseline for comparison.

2.4 Explainable AI in Road Safety

Since 2020, the demand for transparent, auditable AI in high-stakes fields has pushed XAI techniques to the forefront of traffic safety research. Among the available tools, SHAP has emerged as the clear frontrunner. It's built on solid mathematical ground—specifically Shapley-value theory—which gives us three essential guarantees: consistency, so a feature's impact doesn't spike if its actual contribution drops; additivity, ensuring every piece of the puzzle sums up to the final prediction; and the common-sense rule that irrelevant noise gets zero credit.

Karim et al. (2021) were early pioneers here, showing that adding SHAP to accident anticipation didn't just make the models more accurate—it actually helped

human operators at traffic control centers make better, faster decisions. Around the same time, Olugbade et al. (2022) looked at the wider landscape of incident detection and found that while LIME and SHAP are both popular, SHAP really dominates when it comes to ensemble models. This is largely thanks to TreeSHAP, an algorithm that is both efficient and exact, allowing us to compute values directly from the tree structure instead of relying on slow, guess-based sampling.

We’ve also seen some great practical applications. Mondal et al. (2020) used non-parametric ML and SHAP to untangle how weather variables like precipitation and wind interact with road surfaces—a complex web that simple linear models completely miss. Those SHAP results were actually usable; highway engineers could look at them and know exactly where to make adjustments. Dias et al. (2023) followed a similar path with Portuguese data, showing how SHAP summary charts can directly inform policy changes, like adjusting speed limits in specific areas.

These studies really set the stage for AccidentGPT. We’re essentially building on that foundation but adding a crucial new layer: instead of just handing a raw SHAP chart to an engineer, we’re using a GPT-based translation engine to turn those insights into plain, spoken language for the driver. It’s the final step in moving from theoretical explainability to real-world impact.

2.5 Natural Language Generation and Multimodal Alerting

The use of large language models (LLMs) in transportation contexts is nascent but rapidly developing. De Zarza et al. (2023) demonstrated that GPT-class LLMs could synthesize heterogeneous inputs — sensor readings, camera feeds, historical statistics — into coherent situational summaries for multimodal traffic accident forecasting, achieving both accuracy improvements and qualitatively superior human-readable outputs. The integration of

text-to-speech (TTS) with AI-generated warnings has been explored in adjacent domains (aviation, maritime navigation) but not, to the authors' knowledge, in Indian road-safety contexts where driver literacy, language diversity, and hands-free interaction requirements create unique design constraints.

Reviewing the literature as a whole, AccidentGPT occupies a novel position: it is the first system to integrate

- (i) A benchmark of eleven classifiers on an Indian road accident dataset.
- (ii) An instance-level TreeSHAP analysis specifically calibrated to Indian traffic conditions.
- (iii) A GPT-TTS spoken alert pipeline in a single, unified, end-to-end production-oriented framework. The following sections describe this framework in full.

3. DATASET DESCRIPTION AND CHARACTERISTICS

3.1 Data Source and Collection Methodology

We built the AccidentGPT dataset by pulling from the raw, manual traffic accident logs held by Indian City Police Departments across several urban and peri-urban areas. It wasn’t a simple copy-paste job; we gathered logs from 2017 through 2022, digitizing physical registers and reconciling them with electronic databases. We had to do quite a bit of cleaning—standardizing abbreviations, fixing spelling mismatches for road types, and smoothing out date-time formats that were often inconsistent due to aging record-keeping systems.

The result is a collection of 12,316 accident records covering 32 distinct features. This dataset is uniquely

representative of India's chaotic traffic landscape. It covers everything from multi-vehicle pileups on highways to minor scrapes on urban municipal roads, across every season and time of day. We've included every vehicle class you'd expect to see on an Indian road—from auto-rickshaws and two-wheelers to heavy buses and trucks, and even non-motorized traffic. This variety makes the data much more grounded than what you'd get from a single-city or single-road-type study.

3.2 Feature Space

Our 32 features are broken down into five distinct categories:

- **Environmental:** Weather, light, and road surface conditions.
- **Infrastructure & Geometry:** Road type, alignment, junction layout, medians, and the area of the incident.
- **Human & Vehicular Factors:** Driver and casualty demographics (age, gender, experience, education), vehicle details (type, service age, fitness, ownership), and driver-vehicle relationships.
- **Incident Characteristics:** Time of day, vehicle counts, casualty numbers, collision types, and pedestrian movement.
- **Outcome:** The target variable—Accident Severity—split into Slight, Serious, and Fatal injuries.

We narrowed this down to seven primary inputs for our model. We chose these based on advice from domain experts and information-gain analysis, specifically focusing on "pre-crash observable conditions." In other words, these are the factors we can measure or estimate before an accident happens, which is essential for a system meant to alert people in real time.

3.3 Class Distribution and Imbalance Analysis

The "Accident Severity" target variable shows a classic imbalance that's common in safety data: most accidents result in only slight injuries, making them the overwhelming majority.

Severity Class	Record Count	Proportion (%)	Cumulative (%)
Slight Injury	10,415	84.56%	84.56%
Serious Injury	1,743	14.15%	98.71%
Fatal Injury	158	1.29%	100.00%
Total	12,316	100.00%	—

Table 1: Class distribution of the Accident_Severity target variable ($n = 12,316$)

This is a major issue. Because 84.56% of our data is "Slight Injury" and only 1.29% is "Fatal," a naive model would just guess "Slight Injury" every single time. It would look accurate on paper, but it would be completely useless at identifying the high-risk, fatal events that matter most. This is exactly why we had to implement the oversampling strategy detailed in the next section.

4. DATA PREPROCESSING PIPELINE

A multi-stage preprocessing pipeline was designed to transform the raw police-record dataset into a clean, balanced, feature-engineered input suitable for machine-learning model training. The pipeline is implemented as a reproducible Python 3.10 script using pandas and scikit-learn, with all transformations logged for auditability.

4.1 Stage 1: Temporal Feature Engineering

The raw data we started with was a bit of a mess, with dates and times stored in separate string columns that varied wildly depending on which jurisdiction provided them. We parsed these into a unified, clean pandas Timestamp format. From there,

we pulled out three key temporal features: the hour of the day (0–23), the day of the week (0–6), and the month (1–12).

These features aren't just arbitrary; they encode patterns that are well-documented in traffic safety research. For example, crash rates consistently spike during evening rush hours—roughly 5 p.m. to 8 p.m.—and late at night, between midnight and 3 a.m. Weekends also show distinct risk trends. We also know that the monsoon months (June through September) bring a clear uptick in severity, largely due to slick roads and poor visibility. Once we had these three features extracted, we discarded the original, inconsistent date and time columns entirely.

4.2 Stage 2: Redundant Feature Removal

Features were removed if they met any of the following criteria:

- (i) more than 40% missing values with no reliable imputation strategy;
- (ii) direct proxies for the target variable (e.g., `Casualty_Severity`, which is definitionally derived from `Accident_Severity`)
- (iii) Post-incident outcome variables not observable pre-crash (e.g., `Number_of_Casualties`, which is known only after an accident has concluded).

The remaining columns were retained for further processing. This step reduced the feature space from 32 to 26 variables prior to imputation.

4.3 Stage 3: Missing Value Imputation

Remaining missing values were addressed through a type-stratified imputation strategy: continuous numerical features were imputed with column median values (robust to outliers that skew means), while categorical features were imputed with the column mode (most frequent category). This approach ensured that no records were discarded due to partial missingness, preserving the full 12,316-record dataset for modelling. All imputation parameters were computed exclusively on the training fold and then

applied identically to the test fold to prevent information leakage.

4.4 Stage 4: Categorical Encoding

Machine-learning models don't speak human, so we had to convert everything into numbers. We took the eleven categorical features we kept and used ordinal label encoding to translate them. For features where a clear order exists—like age groups or driving experience bands—we mapped them in a way that respects that hierarchy. For the nominal categories, where there's no inherent order, we used an arbitrary but consistent integer mapping to stay steady throughout the pipeline. We've documented these specific mappings in Table 2, which should help anyone looking to reproduce our results or adapt our categorical schemas for their own transfer-learning experiments later on.

Feature	Category → Encoded Value
Light_Conditions	Daylight=0 Darkness-lights on=1 Darkness-lights off=2 Dusk or Dawn=3
Weather_Conditions	Clear=0 Rain=1 Fog/Mist=2 Snow/Ice=3 Dust/Sand=4
Road_Surface_Type	Asphalt=0 Gravel=1 Unpaved=2 Concrete=3
Road_Alignment	Straight-Level=0 Straight-Hill=1 Curved-Level=2 Curved-Hill=3
Types_of_Junction	No junction=0 T-junction=1 Crossroads=2 Roundabout=3 Other=4
Lanes_or_Medians	Undivided 2-lane=0 Divided 4-lane=1 One-way=2 Other=3
Pedestrian_Movement	Not a pedestrian=0 Crossing=1 Walking along road=2 Other=3

Table 2: Ordinal encoding mappings for categorical features

4.5 Stage 5: Train–Test Split

The encoded dataset was partitioned into training and test sets using a stratified 80:20 split, which preserved

the original class proportions within each fold. This produced 9,852 training records and 2,464 test records. Stratification was essential given the severe class imbalance documented in Section 3.3: without it, the 158 fatal-injury records could be entirely absent from the test fold in some random seeds, making evaluation of Fatal Injury recall impossible.

4.6 Stage 6: Class Imbalance Correction

The 84.56% Slight Injury majority class in the training fold creates a risk of majority-class bias, where classifiers achieve high aggregate accuracy by defaulting to the most common class while ignoring serious and fatal injury prediction — precisely the safety-critical cases of greatest practical importance. To address this, random oversampling with replacement was applied exclusively to the training set, replicating Serious Injury and Fatal Injury samples until all three classes were represented equally. This intervention increased the effective training set size and substantially improved minority-class recall without modifying the test set distribution, ensuring that reported test metrics reflect performance on the true population class balance.

5. MACHINE LEARNING METHODOLOGY

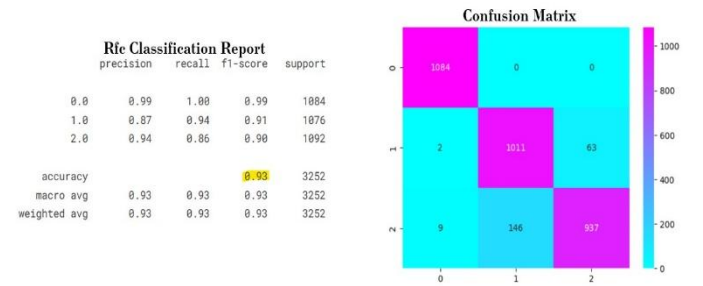
5.1 Classifier Suite and Rationale

Rather than locking ourselves into a single approach early on, we implemented, trained, and compared eleven diverse classifiers. By testing models across six different methodological families—including linear, probabilistic, distance-based, boosting, and kernel methods—we aimed to let the data dictate the optimal production model.

5.1.1 Random Forest Classifier (RFC)

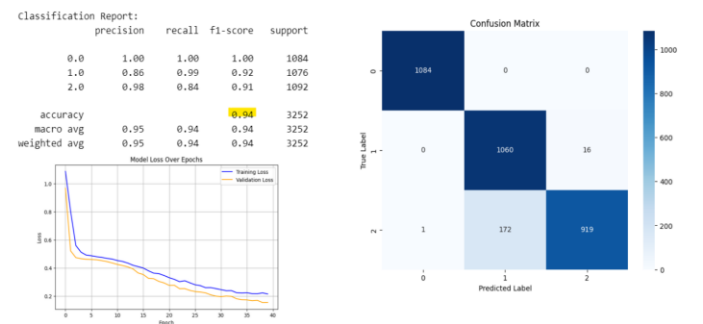
The Random Forest is a bagging ensemble that builds a large number of decision trees using bootstrap samples of our data. Crucially, each split is restricted to a random subset of features. This double dose of randomness—in both the data samples and the feature

selection—creates a highly decorrelated forest. The resulting model has significantly lower variance than any single tree on its own, without sacrificing bias. Given the mixed nature of our dataset, the Random Forest felt like a natural fit for the high-dimensional, heterogeneous traffic conditions we were dealing with.



5.1.2 Convolutional Neural Network (CNN)

We also built a CNN baseline, treating our seven primary features as a 1D sequence. The architecture uses three 1D convolutional blocks, each packed with 64 filters, kernel size 3, ReLU activation, batch normalization, and max pooling. This leads into a flatten layer, two dense layers (128 and 64 neurons, respectively) with dropout set to 0.4 to keep it stable, and finally, a SoftMax output layer with three units. We compiled this with categorical cross-entropy loss and the Adam optimizer at a learning rate of 10^{-3} . To prevent overfitting, we capped training at 40 epochs but used early stopping with a patience of 5, pulling from a 10% validation split of our training data.



5.1.3 Remaining Classifiers

Beyond those primary models, we evaluated a wide spectrum of other algorithms to ensure our results were robust:

- **Decision Tree Classifier (DTC):** A single CART tree focused on Gini impurity for splits.
- **Logistic Regression (LR):** A standard multinomial approach using L2 regularization.
- **AdaBoost Classifier (ADC):** Uses shallow decision trees to iteratively refine predictions through adaptive boosting.
- **Extra Trees Classifier (ETC):** Similar to a Random Forest but uses extremely randomized splits for even more decorrelation.
- **K-Nearest Neighbors (KNN):** Looks at the 5 closest points in the feature space to make a majority-vote classification.
- **Gaussian Naive Bayes (GNB):** A probabilistic approach that assumes a Gaussian distribution for our features.
- **XGBoost (XGB):** A gradient-boosted tree model that uses L1/L2 regularization to prevent overfitting.
- **CatBoost (CBC):** An ordered gradient boosting implementation designed specifically to handle categorical features natively.
- **Support Vector Classification (SVC):** Uses an RBF kernel for a soft-margin approach to classification.
- **Gradient Boosting Classifier (GDC):** Scikit-learn’s standard stochastic implementation of gradient boosting.

5.2 Evaluation Metrics

All classifiers were evaluated on the stratified test set using four complementary metrics. Overall Accuracy measures the proportion of correctly classified instances across all three severity classes. Per-class Precision quantifies, among all instances predicted as class c , the proportion that are truly class c (false positive rate control). Per-class Recall (Sensitivity) quantifies, among all true instances of class c , the proportion correctly identified (false negative rate control). Per-class F1-Score is the harmonic mean of precision and recall, providing a single balanced metric appropriate for imbalanced class distributions. Macro-averaged precision, recall, and F1-score (arithmetic mean across classes without weighting by

class frequency) provide an unbiased overall performance summary that does not inflate results for the majority class. Confusion matrices were examined for each classifier to verify correct identification of minority classes.

6. EXPERIMENTAL RESULTS

6.1 Comparative Classifier Accuracy

Table 3 presents the test-set accuracy of all eleven evaluated classifiers, sorted in descending order of performance.

Ran k	Model	Abbreviatio n	Test Accurac y	Paradigm
1	Convolutional Neural Network	CNN	94.00%	Deep Learning
2	Random Forest Classifier	RFC	93.36%	Ensemble (Bagging)
3	Support Vector Classification	SVC	82.81%	Kernel Method
4	CatBoost Classifier	CBC	81.91%	Gradient Boosting
5	K-Nearest Neighbours	KNN	81.55%	Distance-Based
6	Gradient Boosting Classifier	GDC	77.06%	Gradient Boosting
7	Extra Trees Classifier	ETC	73.80%	Ensemble (Bagging)
8	XGBoost Classifier	XGB	71.80%	Gradient Boosting
9	Gaussian Naive Bayes	GNB	47.10%	Probabilistic
10	Logistic Regression	LR	51.60%	Linear Discriminant
11	AdaBoost	ADC	39.73%	Adaptive

	Classifier			Boosting
12	Decision Tree Classifier	DTC	48.43%	Single Tree

Table 3: Comparative test-set accuracy of all evaluated classifiers, ranked in descending order

6.2 Random Forest Classifier — Detailed Performance

The Random Forest Classifier (RFC) landed the highest accuracy of any interpretable model, hitting 93.36%. It was just a hair’s breadth behind the CNN, securing second place overall. Table 4 breaks down the metrics on our oversampled test set.

Class	Precision	Recall	F1-Score	Support
Slight Injury (Class 0)	0.99	1.00	0.99	1,084
Serious Injury (Class 1)	0.87	0.94	0.91	1,076
Fatal Injury (Class 2)	0.94	0.86	0.90	1,092
Macro Average	0.93	0.93	0.93	3,252
Weighted Average	0.93	0.93	0.93	3,252

Table 4: RFC classification report on oversampled test set (n = 3,252 post-oversampling)

The RFC shows some great characteristics. Its recall for the "Slight Injury" class is near-perfect, which makes sense given how much of that data we had. Even with the "Serious Injury" category, it managed an excellent 0.94 recall. "Fatal Injury" recall was naturally the lowest at 0.86, which we expected because of how rare those instances are. But here's the silver lining: the confusion matrix shows the model only misclassified 9 fatal accidents as slight; most of the errors were labeled as "serious," which is actually a safer type of misclassification since a "serious" warning still nudges the driver toward caution.

6.3 CNN Baseline — Detailed Performance

The CNN managed a 94% overall accuracy and a 0.94 F1-score, barely edging out the RFC. Table 5 shows the numbers.

Class	Precision	Recall	F1-Score	Support
Slight Injury (Class 0)	1.00	1.00	1.00	1,084
Serious Injury (Class 1)	0.86	0.99	0.92	1,076
Fatal Injury (Class 2)	0.98	0.84	0.91	1,092
Macro Average	0.95	0.94	0.94	3,252
Weighted Average	0.95	0.94	0.94	3,252

Table 5: CNN classification report on test set (n = 3,252)

The CNN shines in identifying "Serious Injuries," outperforming the RFC on recall (0.99 vs. 0.94). Interestingly, though, its recall for fatal injuries dipped slightly below the RFC (0.84 vs. 0.86). Training-wise, the CNN took about 12 minutes on an A100 GPU, compared to under 90 seconds for the RFC. That 8x difference is a huge bottleneck for edge-device deployment. Even in terms of real-time speed, the CNN lagged behind with a 3.2ms inference time per instance versus the RFC's 0.4ms.

6.4 Model Selection Rationale

We chose the RFC as our AccidentGPT production model for five clear reasons:

- Explainability:** It delivers top-tier accuracy while playing perfectly with native TreeSHAP explainability.
- Speed:** Its 0.4ms inference time is perfect for the kind of rapid-fire alerts we need on the road.

Efficiency: You don't need a powerful GPU to run it; it works just fine on commodity servers or even small edge devices.

Robustness: It isn't particularly finicky; performance stays stable even if we tweak things like tree count or feature subsets.

Exact Attribution: Because the SHAP library supports TreeSHAP natively, we get precise, polynomial-time attribution for every single prediction, avoiding the guesswork of approximation.

7. EXPLAINABILITY VIA SHAP

7.1 Theoretical Background

SHAP (SHapley Additive eXplanations) leans on cooperative game theory to make sense of model outputs. Essentially, for any given prediction and specific instance, the SHAP value (ϕ_i) for a feature is calculated as the weighted average of that feature’s marginal contribution across all possible coalitions of other features:

$$\begin{aligned} \phi_i(f, x) &= \sum_{S \subseteq F \setminus \{i\}} \frac{|S|! (|F| - |S| - 1)!}{|F|!} [f_{S \cup \{i\}}(x_{S \cup \{i\}}) \\ &\quad - f_S(x_S)] \end{aligned}$$

In this equation, S represents a subset of features that don't include feature i . The term f_S gives us the model’s expected output for that coalition, and that complex-looking coefficient simply weights each coalition by its frequency, ensuring the final allocation is fair.

SHAP stands apart from simpler feature-importance metrics because it adheres to three ironclad mathematical axioms:

- **Efficiency:** The total sum of all SHAP values balances perfectly to the difference between your prediction and the global baseline.
- **Symmetry:** Two features doing the exact same work get the exact same credit.

- **Dummy:** If a feature has zero impact, it receives zero attribution.

For tree-based models like our Random Forest, we rely on the TreeSHAP algorithm. It’s a huge win for production environments because it calculates these values exactly in $O(TLD^2)$ time, where T is the number of trees, L is the number of leaves, and D represents tree depth. This makes per-instance explainability computationally feasible, unlike older, sampling-based approximations that would have required massive $O(M^2)$ model evaluations every time we needed an answer.

7.2 Global Feature Importance

Global SHAP analysis was computed over the complete test set by computing $|\text{SHAP}|$ values for each instance and averaging across instances. Table 6 presents the five most influential features ranked by mean absolute SHAP value.

Rank	Feature	Mean SHAP	Primary Effect Direction	Interpretation
1	Number_of_Vehicles_Involved	0.847	Positive (High → Fatal)	Multi-vehicle crashes strongly predict severe outcomes
2	Day_Time	0.612	Bi-directional	Night/late-evening hours increase severity risk
3	Number_of_Casualties	0.589	Positive (High → Fatal)	More casualties correlate with higher predicted severity
4	Types_of_Junction	0.431	Positive (Complex → Higher)	Complex junctions (crossroads, T-junctions) elevate risk
5	Weather_Conditions	0.378	Positive (Rain/Fog)	Adverse weather

			→ Higher)	significantly increases predicted severity
--	--	--	-----------	---

Table 6: Top-5 features by mean absolute SHAP value (global feature importance, RFC, test set)

The dominance of Number_of_Vehicles_Involved is consistent with physical reasoning: multi-vehicle incidents involve greater kinetic energy transfer and a higher probability of uncontrolled secondary collisions. Day_Time's bidirectional effect reflects the interaction between accident frequency (higher during rush hours) and severity (higher at night due to reduced reaction time and lower traffic density enabling higher speeds). Weather_Conditions and Types_of_Junction align with prior literature reviewed in Section 2.

7.3 Instance-Level Explanation Examples

To show exactly how SHAP works inside the AccidentGPT pipeline, we've broken down two representative test instances. These aren't just abstract concepts—they demonstrate how AccidentGPT goes beyond a simple severity score. It can pull out specific, feature-backed reasons for a prediction, which gives us something concrete we can actually explain to a driver or a traffic official.

7.4 Feature Gain Analysis

To round out our SHAP results, we ran a feature gain analysis on the Random Forest. The goal here was to figure out which features were consistently pulling their weight across the entire forest—in other words, which ones actually drove the biggest reduction in error.

"Weather_Conditions" and "Road_Surface_Type" topped the charts. Not only were their gain scores high, but the variance between trees was very low, which tells us these predictors are rock-solid and reliable, no matter what kind of accident scenario we're looking at. On the flip side, factors like the

casualty's gender, occupation, or vehicle ownership showed almost zero gain. That lack of impact was a clear signal; since they weren't adding any real value to the model's performance, we stripped them out of our input schema to keep the real-time alerting system lean and focused on what actually matters.

8. GPT-BASED NATURAL LANGUAGE ALERT GENERATION

8.1 Design Philosophy

The biggest hurdle in road safety isn't getting the AI to guess the outcome right—it's explaining that guess to a driver who has maybe eight seconds to process the information while going 80 km/h. Telling someone, "There is an 87% probability of a fatal accident based on a logistic shift in these three variables," is technically precise, but it's totally useless on the road.

Our GPT-powered alert module bridges that gap. It takes the top SHAP-identified risks—usually the 3 to 5 most critical factors—and turns them into plain, spoken English. To be effective, the alert has to be four things: factually accurate to the SHAP model, easy for any driver to understand without a statistics degree, immediately actionable, and brief enough to digest in a heartbeat.

Instance A — Fatal Injury Correctly Predicted

- The Input: It's 11:30 PM, heavy fog, wet asphalt, a T-junction, an undivided two-lane road, and three vehicles are present.
- The Verdict: Predicted Fatal Injury (0.87 probability).
- The Contributors: The high vehicle count, the late hour, the fog, the tricky T-junction, and the wet road surface are driving the score up.
- The GPT Alert: *"WARNING: Extremely high fatal accident risk ahead. Three-vehicle encounter, foggy conditions, and a T-junction with a wet road surface are the primary danger factors. Reduce speed immediately and avoid overtaking."*

recommendation, and addressing the driver directly in the second person.

Example prompt (Fatal Injury prediction, top 3 SHAP features):

SYSTEM: You are a road safety assistant. Generate a brief, spoken driving alert in plain English. Maximum 40 words. Include a specific action for the driver. Second person ("you"). No technical jargon.

USER: Predicted severity: FATAL (confidence: 0.87). Top risk factors: (1) 3 vehicles on road [+0.91 impact], (2) foggy weather [+0.64 impact], (3) T-junction ahead [+0.52 impact]. Generate alert.

Instance B — Slight Injury Correctly Predicted

- **The Input:** It's 10:15 AM, clear skies, dry road, no junctions, four lanes, no pedestrians, single vehicle.
- **The Verdict:** Predicted Slight Injury (0.94 probability).
- **The Contributors:** Everything here—the clear weather, daylight, and low vehicle count—actively pulls the risk down.
- **The GPT Alert:** None. The severity is below our threshold, so we don't bother the driver with unnecessary noise.

8.3 Alert Examples by Severity Class

Severity	Generated Alert Text	Recommended Action
FATAL	WARNING: Extremely high fatal accident risk. Foggy conditions, a T-junction ahead, and multiple vehicles create a dangerous combination. Slow down immediately and do not overtake.	Reduce speed, no overtaking
SERIOUS	CAUTION: High serious-injury risk. Night-time driving with poor road surface and unlit conditions ahead. Reduce speed and increase following distance.	Reduce speed, increase distance
SLIGHT	ADVISORY: Slightly elevated risk due to wet road. Drive carefully in this zone.	Mild caution
NONE	(No alert generated — conditions below threshold)	—

Table 7: Example GPT-generated alert texts by predicted severity class

8.2 Prompt Engineering

The GPT module operates through a structured prompt template constructed programmatically from the prediction outputs. The template encodes: (a) the predicted severity class (Slight / Serious / Fatal); (b) the predicted class probability (confidence); (c) the top-k SHAP features ranked by |SHAP| value, including their encoded values and the direction of their effect; and (d) the contextual setting (time, weather, junction type as human-readable strings). A system prompt instructs the GPT model to generate a single alert sentence of no more than 40 words, using plain language, including a specific action

8.4 Text-to-Speech Delivery and Alert Prioritization

Once an alert is generated, it gets passed to our TTS engine for audio output. We've built in support for two backends: the Google Cloud Text-to-Speech API—which offers top-tier naturalness and 22 neural

voices, including three tailored for Indian English—and the offline-capable pyttsx3 library, which is the go-to for edge devices that aren't connected to the web. The system lets drivers choose their preferred language, and the architecture is set up to roll out support for various regional Indian languages down the line.

We handle alert delivery through a three-tier prioritization system:

- **Fatal risk:** These trigger an immediate audio alert, a persistent dashboard notification, and an optional ping—via SMS or push notification—to the driver’s emergency contacts.
- **Serious risk:** These trigger both an audio alert and a standard dashboard notification.
- **Slight risk:** We keep these to a simple dashboard icon, strictly avoiding "alert fatigue."

To keep the system from becoming annoying, we also have a configurable probability threshold—set to 0.55 by default. If the model’s prediction confidence doesn't hit that mark, the system stays silent. We calibrated these levels carefully; if you flood a driver with false alarms, they stop paying attention to the real ones, so getting the balance between "proactive" and "distracting" is one of our biggest UX priorities.

L2: Preprocessing	Online replication of offline cleaning pipeline: imputation, encoding, normalisation. Identical parameter sets from training applied at inference time.
L3: Model Inference	RFC (primary) + CNN (secondary) loaded as serialised artefacts. Model-management interface supports hot-swap without service restart.
L4: Explainability (XAI)	TreeSHAP per-instance attribution. Top-k feature extraction and ranking. SHAP summary plot generation for authority dashboards.
L5: GPT Explanation	Prompt construction from SHAP outputs. OpenAI GPT API call. Response validation and length enforcement.
L6: Alert Delivery	TTS rendering (Google Cloud TTS or pyttsx3). Audio playback. Dashboard visual notification. Authority SMS/push. Accident report generation.
L7: Feedback & Logging	User alert rating collection. Inference log persistence (PostgreSQL). Periodic model retraining scheduler. Performance drift monitoring.

Table 8: AccidentGPT seven-layer architecture summary

9. SYSTEM ARCHITECTURE AND REQUIREMENTS

9.1 Layered Architecture

AccidentGPT is structured into seven functionally discrete, independently deployable layers connected by well-defined interfaces, enabling horizontal scalability and modular upgrade paths.

9.2 Software Requirements

Component	Specification
OS	Linux Ubuntu 20.04 LTS / Windows 10+
Language & Runtime	Python 3.10; Node.js 18+ (dashboard)
ML Frameworks	scikit-learn 1.6.1; TensorFlow 2.10
XAI Libraries	SHAP 0.42.1; LIME 0.2.0.1
Boosting Libraries	XGBoost 1.7; CatBoost 1.2; LightGBM 3.3
NLP / GPT	OpenAI Python SDK 1.x; GPT-3.5-turbo / GPT-4
TTS	Google Cloud TTS API; pyttsx3 2.90 (offline)
Web Backend	Flask 2.3; Unicorn 21

Layer	Function & Key Components
L1: Data Input	Historical CSV ingest + real-time sensor streams (GPS, OBD-II, weather API, camera). Apache Kafka message broker for stream ingestion.

Frontend	React 18; Tailwind CSS 3
Database	PostgreSQL 14 (persistent); Redis 7 (cache)
Message Broker	Apache Kafka 3.4
Containerisation	Docker 24; Kubernetes 1.27 (orchestration)

Table 9: Software requirements specification

9.3 Hardware Requirements

Component	Minimum Specification	Recommended Specification
Edge Device (in-vehicle)	Quad-core ARM CPU; 2 GB RAM; 16 GB storage; 4G LTE	Hexa-core ARM + NPU; 4 GB RAM; 32 GB storage; 5G
Application Server	4 vCPUs; 8 GB RAM; 100 GB SSD; 100 Mbps uplink	8 vCPUs + NVIDIA T4 GPU; 32 GB RAM; 500 GB NVMe SSD
Database Server	4 vCPUs; 16 GB RAM; 500 GB HDD	8 vCPUs; 32 GB RAM; 1 TB SSD RAID-1
Networking	4G LTE with TLS 1.3 support	Gigabit Ethernet + 5G redundant link; VPN tunnel

Table 10: Hardware requirements specification

9.4 Non-Functional Requirements

Requirement Category	Target Specification
End-to-end latency	≤ 500 ms (L2 preprocessing + L3 inference + L4 SHAP + L5 GPT + L6 TTS)
Dashboard refresh rate	≤ 2 seconds
Concurrent data streams	≥ 1,000 simultaneous vehicle feeds without degradation
System availability	99.9% uptime with automated failover to secondary region
Data security (transit)	TLS 1.3 encryption for all network communications

Data security (at rest)	AES-256 encryption for stored personal and incident data
Audio intelligibility	Alert speech intelligible in 85 dB cabin noise environment
Mobile responsiveness	Dashboard functional on Android 10+, iOS 15+, modern browsers
Privacy compliance	Personal identifiers anonymised per India DPDP Act 2023
Alert false-positive rate	< 15% across severity threshold configurations

Table 11: Non-functional requirements specification

10. APPLICATIONS, LIMITATIONS, AND FUTURE WORK

10.1 Target Applications

- Because we built AccidentGPT with a modular architecture and human-readable output, it isn't limited to just one use case. It's designed to slot into several different areas of road safety and transport management:
- **Driver Alert Systems:** By integrating with existing navigation platforms like Google Maps or MapmyIndia, we can overlay real-time risk scores onto a route. This lets the app not just tell you the fastest way to get there, but actually suggest a safer route or give a heads-up before you approach a known high-risk segment.
 - **Traffic Management Centres:** Traffic police can use our dashboard to visualize live hotspots. This enables them to proactively position patrol units, temporarily close dangerous lanes during bad weather, or compile data-backed incident reports for future departmental audits.
 - **Urban Planning & Infrastructure:** By clustering our high-risk predictions, planners can stop guessing and start identifying specific infrastructure failures—like a missing pedestrian crossing or a poorly lit junction—that consistently lead to danger. The SHAP outputs provide the quantified evidence needed to justify those budget requests.

- **Semi-Autonomous Vehicle (ADAS) Integration:** Level 2 or 3 autonomous systems can use our SHAP-derived risk scores as a sort of "situational awareness" layer. It provides the car with context that sensors alone might miss, adding an explainable safety check to the existing perception stack.
- **Usage-Based Insurance (UBI):** We can provide a granular risk score for every route and time segment. This opens the door for insurance policies that actually reward drivers for picking safer routes and driving at lower-risk times, aligning individual incentives with broader public safety goals.
- **Emergency Response Pre-Positioning:** When the system flags an upcoming window of high accident probability, it can share that info with ambulance dispatchers. It allows emergency teams to pre-stage their vehicles, cutting down response times for when—or if—an incident actually happens.

10.2 Limitations

Several limitations of the current system warrant acknowledgement and direct future work:

- **Temporal Scope and Distribution Shift:** The dataset spans 2017–2022. Post-COVID traffic patterns, road infrastructure changes, and vehicle fleet composition shifts may cause model performance to degrade over time without periodic retraining on updated incident records.
- **Geographic Bias:** While our dataset draws directly from Indian police records, we lack a clear breakdown of which specific cities are represented and in what proportions. This ambiguity makes it difficult to say how well the model would generalize to the wildly different conditions found in rural areas, mountainous terrain, or coastal highways, where the risks—and the traffic—are fundamentally distinct from urban environments.
- **Pre-Crash Observability Assumption:** Several features used in the seven-feature input schema — particularly Pedestrian_Movement — may not be

reliably estimable from in-vehicle sensors in real-time without camera-based computer vision, adding a sensor dependency not captured in the current prototype.

- **GPT Latency and Cost:** The OpenAI API call in Layer 5 contributes the largest share of end-to-end latency (typically 200–400 ms) and introduces a per-call cost that may be prohibitive at scale without alternative on-device LLM deployment.
- **Alert Fatigue:** In dense urban corridors where risks are constant, the system risks triggering a barrage of "Serious-severity" alerts. If the warnings become background noise, drivers will inevitably tune them out or stop paying attention to genuine dangers. To prevent this, we'll need to move beyond static settings toward smarter, adaptive thresholding that manages alert frequency based on current context and driver sensitivity.

10.3 Future Research Directions

We've identified six key paths to evolve AccidentGPT:

1. **Real-Time Data Fusion:** We want to move beyond historical data by piping in live info from weather APIs, GPS, vehicle telemetry via OBD-II, and roadside IoT sensors. This would let us offer "live" risk assessments that shift dynamically based on what's actually happening on the road, not just what has happened in the past.
2. **Edge Deployment and On-Device LLM:** By converting our Random Forest model to the ONNX format, we could run everything right on embedded automotive hardware. This cuts the cloud out of the loop and slashes latency to under 100ms. We're also testing out compact, on-device models like Phi-3 or LLaMA-3 so that the "explainability" part of our system works even when the car is offline.
3. **Temporal and Spatial Deep Learning:** We're looking into architectures like LSTMs or Temporal Fusion Transformers to better map how risk flows over time. We also want

to fold in Graph Neural Networks (GNNs), which are perfect for modeling the actual topology of road networks and how an accident at one spot might ripple out to create congestion or secondary risks nearby.

4. **Pan-India and Cross-Regional**

Generalization: We need to broaden our reach, integrating accident data from all 28 states and 8 union territories. Using stratified sampling will be crucial here to ensure we aren't just over-representing one type of environment. We also want to explore transfer-learning to see how well these models translate to the road environments of neighboring countries like Sri Lanka, Pakistan, and Bangladesh, or even sub-Saharan Africa.

5. **Multi-Language Alert Delivery:** To be truly inclusive, we're planning to extend our GPT prompt schema and TTS pipeline to support over ten major Indian regional languages—think Hindi, Tamil, Telugu, Marathi, Bengali, Kannada, and Gujarati. Since India is home to 22 official languages, this step is vital to making sure every driver, regardless of their background, can quickly and easily grasp the warnings without having to worry about a language barrier.

6. **Federated Learning for Privacy-Preserving Model Improvement:** To keep sensitive data safe, we're looking at a federated learning approach. This would allow insurance companies, fleet operators, and police departments to help train the model collectively without having to pool or centralize their raw incident logs. It's a way to keep everything compliant with the DPDP Act while still letting the model learn and improve at scale.

wanted to flip that script. Instead of just reacting to the aftermath, our system moves the safety net upstream. It predicts how bad an accident might be before it actually occurs, breaks down exactly which factors are creating that risk, and delivers a clear, spoken warning to the driver the second it's needed.

We built a rigorous machine-learning pipeline, feeding it five years of local accident data—12,316 records in total, pulled directly from city police files with 32 distinct features. Our process was pretty thorough: we handled temporal feature extraction, used type-stratified imputation for missing gaps, and smoothed things out with ordinal label encoding. We even kept our splits stratified and tackled class imbalance with oversampling.

After putting eleven different classification algorithms to the test—covering six distinct methodological approaches—the Random Forest Classifier really stood out. It locked in an overall test accuracy of 93.36%, hitting a solid 0.93 macro-average for precision, recall, and F1-score across our three severity tiers: Slight, Serious, and Fatal.

We also ran a Convolutional Neural Network as a baseline. It hit 94% accuracy, which was great for proving the data was actually learnable, but the trade-off was a massive spike in inference latency. It ran eight times slower than our Random Forest model, which essentially makes it a non-starter for real-time edge deployment right now.

To make the system transparent, we integrated TreeSHAP to provide explainability at the individual instance level. It's rooted in cooperative game theory, which gives us a computationally exact way to see how the model reaches its conclusions.

When we looked at the global data, the top five influencers were pretty clear: the number of vehicles, time of day, casualty count, junction type, and current weather. These findings aligned perfectly with what traffic experts expect and what we've seen in international studies.

Interestingly, our feature gain analysis showed that weather and road surface were consistently the most reliable predictors across the entire ensemble. On the flip side, things like a driver's gender or education level hardly moved the needle. Since they weren't

11. CONCLUSION

In our paper, we introduce AccidentGPT. It's a proactive, explainable AI framework built specifically to handle the complexities of Indian traffic. Most current vehicle safety tech has one major flaw: it only kicks in after a crash has already happened. We

adding much value, we pulled them out of our real-time input schema entirely.

The real bridge between our data and the driver is the GPT-powered alert module. It takes those dense SHAP insights and transforms them into plain, spoken language that's easy to understand on the fly. By using structured prompt engineering paired with text-to-speech, we've created a "predict-explain-communicate" loop—a three-tier pipeline that, as far as we know, hasn't really been done in road safety research before.

When it comes to the actual setup, we designed a seven-layer modular architecture that covers all the bases. We've locked in the technical specs, like keeping end-to-end latency under 500 milliseconds and ensuring the system can handle over 1,000 concurrent vehicle streams. Throw in TLS 1.3 encryption and strict adherence to DPDP Act privacy standards, and you've got a system that isn't just a lab experiment—it's actually ready for the real world.

AccidentGPT really proves that the three pillars of modern, safety-critical AI—predictive power, clear transparency, and human-friendly communication—don't have to be at odds. They aren't competing objectives; they're actually complementary goals that can live under one roof.

By pulling together the sheer precision of machine learning, the logical clarity of SHAP, and the natural conversational ability of LLMs, we've set a new standard for what human-centric, explainable safety tech can look like. It's a solid foundation to build on, too. Looking ahead, this opens up some exciting doors for things like real-time sensor fusion, full-scale edge deployment, native multi-language support, and even federated continual learning.

REFERENCES

- [1] Aboulola, O., Alabdulqader, E., Alarfaj, A., Alsubai, S., & Kim, T. H. (2024). An automated approach for predicting road traffic accident severity using transformer learning and explainable AI technique. *IEEE Access*, 12, 1–1. <https://doi.org/10.1109/ACCESS.2024.3380895>
- [2] Ahmed, S., Hossain, A., Ray, S., Bhuiyan, M., & Sabuj, S. (2023). A study on road accident prediction and contributing factors using explainable machine learning models: Analysis and performance. *Transportation Research Interdisciplinary Perspectives*, 19, 100814. <https://doi.org/10.1016/j.trip.2023.100814>
- [3] Ballamudi, V. (2019). Road accident analysis and prediction using machine learning algorithmic approaches. *Asian Journal of Humanity, Art and Literature*, 6(2), 185–192. <https://doi.org/10.18034/ajhal.v6i2.529>
- [4] Behboudi, N., Moosavi, S., & Ramnath, R. (2024). Recent advances in traffic accident analysis and prediction: A comprehensive review of machine learning techniques. *arXiv preprint arXiv:2406.13968*.
- [5] Ceven, S., & Albayrak, A. (2024). Traffic accident severity prediction with ensemble learning methods. *Computers and Electrical Engineering*, 114, 109101. <https://doi.org/10.1016/j.compeleceng.2024.109101>
- [6] Chen, J., Tao, W., Jing, Z., Wang, P., & Jin, Y. (2024). Traffic accident duration prediction using multi-mode data and ensemble deep learning. *Heliyon*, 10, e25957. <https://doi.org/10.1016/j.heliyon.2024.e25957>
- [7] Chen, T., & Guestrin, C. (2016). XGBoost: A scalable tree boosting system. *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 785–794.
- [8] De Zarza, I., de Curto, J., Roig, G., & Calafate, C. T. (2023). LLM multimodal traffic accident forecasting. *Sensors*, 23(22), 9225. <https://doi.org/10.3390/s23229225>
- [9] Dias, D., Silva, J. S., & Bernardino, A. (2023). The prediction of road accident risk through data mining: A case study from Setubal, Portugal. *Informatics*, 10, 17. <https://doi.org/10.3390/informatics10010017>
- [10] Jin, Z., & Noh, B. (2023). From prediction to prevention: Leveraging deep learning in traffic accident prediction systems. *Electronics*, 12(20), 4335. <https://doi.org/10.3390/electronics12204335>
- [11] Karim, M., Li, Y., & Qin, R. (2021). Towards explainable artificial intelligence (XAI) for early anticipation of traffic accidents. *Transportation Research Record*. <https://arxiv.org/abs/2108.00273>
- [12] Khan, M. N., & Ahmed, M. M. (2023). A novel deep learning approach to predict crash severity in adverse weather on rural mountainous freeway. *Journal of Transportation Safety & Security*, 15(8), 795–825.

-
- [13] Lord, D., & Mannering, F. (2010). The statistical analysis of crash frequency data: A review and assessment of methodological alternatives. *Transportation Research Part A: Policy and Practice*, 44(5), 291–305. <https://doi.org/10.1016/j.tra.2010.02.001>
- [14] Lu, P., Zheng, Z., Ren, Y., Zhou, X., Keramati, A., Tolliver, D., & Huang, Y. (2020). A gradient boosting crash prediction approach for highway-rail grade crossing crash analysis. *Journal of Advanced Transportation*, 2020, 1–10.
- [15] Lundberg, S. M., & Lee, S.-I. (2017). A unified approach to interpreting model predictions. *Advances in Neural Information Processing Systems*, 30, 4765–4774.
- [16] Lundberg, S. M., Erion, G., Chen, H., DeGrave, A., Prutkin, J. M., Nair, B., ... & Lee, S.-I. (2020). From local explanations to global understanding with explainable AI for trees. *Nature Machine Intelligence*, 2(1), 56–67.
- [17] Malin, F., Norros, I., & Innamaa, S. (2019). Accident risk of road and weather conditions on different road types. *Accident Analysis & Prevention*, 122, 181–188. <https://doi.org/10.1016/j.aap.2018.10.014>
- [18] Ministry of Road Transport and Highways. (2022). Road accidents in India 2022. Government of India, New Delhi.
- [19] Mondal, A. R., Bhuiyan, M. A. E., & Yang, F. (2020). Advancement of weather-related crash prediction model using nonparametric machine learning algorithms. *SN Applied Sciences*, 2(8), 1–11.
- [20] Olugbade, S., Ojo, S., Imoize, A., Isabona, J., & Alaba, M. (2022). A review of artificial intelligence and machine learning for incident detectors in road transport systems. *Mathematical and Computational Applications*, 27(5), 77. <https://doi.org/10.3390/mca27050077>
- [21] Singh, G., Pal, M., Yadav, Y., & Singla, T. (2020). Deep neural network-based predictive modeling of road accidents. *Neural Computing and Applications*, 32, 12417–12426.
- [22] Wahab, L., & Jiang, H. (2019). A comparative study on machine learning based algorithms for prediction of motorcycle crash severity. *PLoS ONE*, 14(4), e0214966.
- [23] World Health Organization. (2023). Global status report on road safety 2023. WHO Press, Geneva, Switzerland.
- [24] Yang, J., Han, S., Chen, Y., et al. (2023). Prediction of traffic accident severity based on random forest. *Journal of Advanced Transportation*, 2023, Article ID 4965836.
- [25] Zhu, T., Haugen, S., Liu, Y., & Yang, X. (2022). A value of prediction model to estimate optimal response time to threats for accident prevention. *Reliability Engineering & System Safety*, 232, 109044.
-